

01. Cover and Lock State

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

A sealed production documentation bundle for the CMCMS_SIMPLIFIED project. It captures scope, database evidence, backend architecture, frontend behavior, RBAC, smoke verification, locked decisions, and final acceptance for the Admin Audit Log Viewer.

Document Field	Locked Value	Why It Matters
Project	CMCMS_SIMPLIFIED	keeps context stable
Phase	Phase 14	current sealed module
Module	Audit Log Viewer	Admin governance viewer
Output	100-page DOCX	documentation handover

FLOW / DIAGRAM

Need traceability -> Inspect existing tables -> Design read-only viewer -> Seal with smoke proof

DETAILED DESCRIPTION

Detailed explanation: Cover and Lock State is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The document opens as a sealed DS-style production deliverable for CMCMS Phase 14. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

02. Document Doctrine

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Every page follows the user style: step-by-step, table-wise, visual, beginner-to-advanced, and seal-ready. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Need traceability -> Inspect existing tables -> Design read-only viewer -> Seal with smoke proof

DETAILED DESCRIPTION

Detailed explanation: Document Doctrine is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Every page follows the user style: step-by-step, table-wise, visual, beginner-to-advanced, and seal-ready. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

03. Executive Abstract

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

Phase 14 turns existing audit trails into an operational governance console. The module is intentionally narrow: Super Admin can inspect and export audit evidence, while all reads remain zero-write and every source table remains unchanged after verification.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Need traceability -> Inspect existing tables -> Design read-only viewer -> Seal with smoke proof

DETAILED DESCRIPTION

Detailed explanation: Executive Abstract is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Phase 14 adds a Super Admin read-only audit viewer over sealed operational traces. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

04. 100-Page Map

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. This page explains how the document is organized from context to implementation to verification. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Need traceability -> Inspect existing tables -> Design read-only viewer -> Seal with smoke proof

DETAILED DESCRIPTION

Detailed explanation: 100-Page Map is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. This page explains how the document is organized from context to implementation to verification. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not hardcode dropdown values when source tables can provide distinct values.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

05. Phase Timeline

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The implementation moved from introspection to permissions, backend, frontend, smoke, and final seal. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Need traceability -> Inspect existing tables -> Design read-only viewer -> Seal with smoke proof

DETAILED DESCRIPTION

Detailed explanation: Phase Timeline is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The implementation moved from introspection to permissions, backend, frontend, smoke, and final seal. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not use backslash for LIKE escape on this MariaDB setup; use vertical bar.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

06. Problem Statement

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Super Admin needs a reliable, zero-write audit viewer for sensitive operational and identity actions. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Need traceability -> Inspect existing tables -> Design read-only viewer -> Seal with smoke proof

DETAILED DESCRIPTION

Detailed explanation: Problem Statement is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Super Admin needs a reliable, zero-write audit viewer for sensitive operational and identity actions. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

07. Scope Boundary

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The module reads existing audit and history tables only and avoids new write surfaces. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Need traceability -> Inspect existing tables -> Design read-only viewer -> Seal with smoke proof

DETAILED DESCRIPTION

Detailed explanation: Scope Boundary is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The module reads existing audit and history tables only and avoids new write surfaces. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not treat notes as guaranteed JSON; fallback text is required.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

08. Read-Only Doctrine

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

Read-only is the central doctrine. This module must never insert, update, delete, alter, or recursively audit a read/export action. It observes existing evidence only. That is why the smoke test verifies source-table counts before and after the API exercises.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Need traceability -> Inspect existing tables -> Design read-only viewer -> Seal with smoke proof

DETAILED DESCRIPTION

Detailed explanation: Read-Only Doctrine is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The strongest locked rule is no inserts, updates, deletes, alters, or recursive audit writes. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

09. Source Table Inventory

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Five tables form the full audit surface: audit_log, identity history, and three status histories. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Need traceability -> Inspect existing tables -> Design read-only viewer -> Seal with smoke proof

DETAILED DESCRIPTION

Detailed explanation: Source Table Inventory is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Five tables form the full audit surface: audit_log, identity history, and three status histories. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

10. audit_log Schema

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The operational audit table uses audit_id as primary key and notes as compact payload storage. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
audit_id	bigint unsigned PK	canonical id
actor_employee_id	varchar(20)	who did it
action	varchar(60)	what happened
entity_type/entity_id	varchar	target object
notes	varchar(500)	JSON/text payload

FLOW / DIAGRAM
Legacy table -> Column introspection -> Index review -> Canonical API shape

DETAILED DESCRIPTION

Detailed explanation: audit_log Schema is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The operational audit table uses audit_id as primary key and notes as compact payload storage. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

11. user_role_history Schema

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Identity and access changes are stored as role/active-state transitions over user_id keys. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
id	bigint unsigned PK	history row
user_id	bigint unsigned	subject user
from_role/to_role	varchar(30)	role transition
from_active/to_active	tinyint	activation diff
actor_user_id	bigint	who changed it

FLOW / DIAGRAM
Legacy table -> Column introspection -> Index review -> Canonical API shape

DETAILED DESCRIPTION

Detailed explanation: user_role_history Schema is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Identity and access changes are stored as role/active-state transitions over user_id keys. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

12. Job Request History Schema

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Job request lifecycle changes are normalized as from_status to to_status events. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
history_id	bigint unsigned PK	history row
jr_no	int	job request
from_status/to_status	varchar(30)	lifecycle diff
transitioned_by	varchar(7)	actor
reason	varchar(500)	why

FLOW / DIAGRAM
Legacy table -> Column introspection -> Index review -> Canonical API shape

DETAILED DESCRIPTION

Detailed explanation: Job Request History Schema is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Job request lifecycle changes are normalized as from_status to to_status events. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not hardcode dropdown values when source tables can provide distinct values.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

13. Job Card History Schema

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Job card transitions use jc_section_no and status history identity. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
history_id	bigint unsigned PK	history row
jc_section_no	varchar(9)	job card id
from_status/to_status	varchar(30)	lifecycle diff
transitioned_by	varchar(7)	actor
reason	varchar(1000)	why

FLOW / DIAGRAM

Legacy table -> Column introspection -> Index review -> Canonical API shape

DETAILED DESCRIPTION

Detailed explanation: Job Card History Schema is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Job card transitions use jc_section_no and status history identity. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not use backslash for LIKE escape on this MariaDB setup; use vertical bar.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

14. Schedule History Schema

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Schedule transitions are stored separately and included in the transitions union. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
id	bigint unsigned PK	history row
schedule_id	bigint unsigned	schedule target
from_status/to_status	varchar(20)	schedule diff
actor_employee_id	varchar(7)	actor
created_at	datetime(6)	time

FLOW / DIAGRAM
Legacy table -> Column introspection -> Index review -> Canonical API shape

DETAILED DESCRIPTION

Detailed explanation: Schedule History Schema is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Schedule transitions are stored separately and included in the transitions union. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

15. Indexes and Query Fitness

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Existing indexes support time, actor, action, entity, and source-specific lookup patterns. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Legacy table -> Column introspection -> Index review -> Canonical API shape

DETAILED DESCRIPTION

Detailed explanation: Indexes and Query Fitness is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Existing indexes support time, actor, action, entity, and source-specific lookup patterns. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not treat notes as guaranteed JSON; fallback text is required.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

16. Baseline Row Counts

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The smoke baseline proves real data existed before the viewer was sealed. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Rows / Value	Meaning
audit_log	127 at final smoke	All Actions
identity history	17 rows	Identity & Access
transitions	91 + 11 + 7	Lifecycle evidence

FLOW / DIAGRAM

Legacy table -> Column introspection -> Index review -> Canonical API shape

DETAILED DESCRIPTION

Detailed explanation: Baseline Row Counts is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The smoke baseline proves real data existed before the viewer was sealed. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

17. Migration 600

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Phase 14 seeds two permission codes and grants both to Super Admin only. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Rows / Value	Meaning
audit_log	127 at final smoke	All Actions
identity history	17 rows	Identity & Access
transitions	91 + 11 + 7	Lifecycle evidence

FLOW / DIAGRAM

Migration 600 -> Permission seed -> SUPER_ADMIN grant -> Sidebar gate

DETAILED DESCRIPTION

Detailed explanation: Migration 600 is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Phase 14 seeds two permission codes and grants both to Super Admin only. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

CODE / CONFIG EXCERPT

```
INSERT IGNORE INTO permissions
(permission_code, resource, action, description)
VALUES ('audit:read-list','audit','read-list','View Audit Log'),
       ('audit:export','audit','export','Export audit CSV');
```

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

18. Permission Codes

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. audit:read-list and audit:export define viewing versus download capabilities. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Rows / Value	Meaning
audit_log	127 at final smoke	All Actions
identity history	17 rows	Identity & Access
transitions	91 + 11 + 7	Lifecycle evidence

FLOW / DIAGRAM

Migration 600 -> Permission seed -> SUPER_ADMIN grant -> Sidebar gate

DETAILED DESCRIPTION

Detailed explanation: Permission Codes is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. audit:read-list and audit:export define viewing versus download capabilities. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

19. RBAC Matrix

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The locked default is tight: Super Admin can view/export, all other roles cannot. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Role	audit:read-list	audit:export
SUPER_ADMIN	YES	YES
ADMIN/LAB roles	NO	NO
VIEW_ONLY	NO	NO

FLOW / DIAGRAM

Migration 600 -> Permission seed -> SUPER_ADMIN grant -> Sidebar gate

DETAILED DESCRIPTION

Detailed explanation: RBAC Matrix is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The locked default is tight: Super Admin can view/export, all other roles cannot. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

20. Sidebar Gate

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The sidebar entry is visible only when audit:read-list is present. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Migration 600 -> Permission seed -> SUPER_ADMIN grant -> Sidebar gate

DETAILED DESCRIPTION

Detailed explanation: Sidebar Gate is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The sidebar entry is visible only when audit:read-list is present. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

CODE / CONFIG EXCERPT

```
{ label: 'Admin · Audit Log', to: '/audit',
  icon: ScrollText, requires: 'audit:read-list' }
```

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

21. Admin UX Placement

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The viewer belongs in the Admin surface because audit trails are governance-sensitive. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Migration 600 -> Permission seed -> SUPER_ADMIN grant -> Sidebar gate

DETAILED DESCRIPTION

Detailed explanation: Admin UX Placement is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The viewer belongs in the Admin surface because audit trails are governance-sensitive. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Requirement	Super Admin audit visibility	Design boundary
Data	Existing audit/history tables	No schema rewrite
Permission	audit:read-list/export	RBAC gate
Outcome	Safe governance surface	Phase seal

TECHNICAL DEPTH

Technical depth: the early phase pages establish why the audit viewer exists before discussing code. This prevents scope creep. The important boundary is that an audit viewer is not a dashboard, not a report editor, and not an admin mutation screen. It is an evidence reader with export capability. That distinction keeps security and implementation clean.

LOCKED NOTE / MEMORY

Locked note: Do not use backslash for LIKE escape on this MariaDB setup; use vertical bar.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Confirm the module remains Super Admin-only.
- Keep the sidebar permission gate aligned with migration 600.
- Do not remove historical audit_log:read until a planned cleanup phase.
- Preserve the read-only governance interpretation.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

22. Backend Architecture

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Backend uses Express routes, validators, controller, service, repo, and database pool. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Backend Architecture is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Backend uses Express routes, validators, controller, service, repo, and database pool. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

23. Route Table

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. All endpoints are GET; export is still GET because it produces a read-only CSV representation. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Endpoint/Schema	Permission	Purpose
GET /audit	audit:read-list	paginated list
GET /audit/filters	audit:read-list	dropdown values
GET /audit/export	audit:export	CSV download
GET /audit/:id	audit:read-list	row detail

FLOW / DIAGRAM
Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Route Table is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. All endpoints are GET; export is still GET because it produces a read-only CSV representation. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

CODE / CONFIG EXCERPT

GET /audit	-> audit:read-list
GET /audit/filters	-> audit:read-list
GET /audit/export	-> audit:export
GET /audit/:id	-> audit:read-list

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

24. Validator Layer

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Zod schemas allow only query/path inputs and reject unknown fields. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Endpoint/Schema	Permission	Purpose
GET /audit	audit:read-list	paginated list
GET /audit/filters	audit:read-list	dropdown values
GET /audit/export	audit:export	CSV download
GET /audit/:id	audit:read-list	row detail

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Validator Layer is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Zod schemas allow only query/path inputs and reject unknown fields. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /.id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

25. Source Tabs Model

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Three source tabs map to three SQL shapes while exposing one canonical API output. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Real table/query	API meaning
audit_log	audit_log SELECT	All Actions
identity	user_role_history SELECT	Identity & Access
transitions	UNION ALL histories	Status Transitions

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Source Tabs Model is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Three source tabs map to three SQL shapes while exposing one canonical API output. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

26. Repository Doctrine

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The repo is the only place that knows true table column names and aliases them consistently. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Real table/query	API meaning
audit_log	audit_log SELECT	All Actions
identity	user_role_history SELECT	Identity & Access
transitions	UNION ALL histories	Status Transitions

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Repository Doctrine is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The repo is the only place that knows true table column names and aliases them consistently. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

27. audit_log Query

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The All Actions tab reads audit_log with actor and employee-name resolution. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Real table/query	API meaning
audit_log	audit_log SELECT	All Actions
identity	user_role_history SELECT	Identity & Access
transitions	UNION ALL histories	Status Transitions

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: audit_log Query is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMC MIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The All Actions tab reads audit_log with actor and employee-name resolution. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

28. Identity Query

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The Identity & Access tab resolves user_id through users.employee_id into employee names. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Real table/query	API meaning
audit_log	audit_log SELECT	All Actions
identity	user_role_history SELECT	Identity & Access
transitions	UNION ALL histories	Status Transitions

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Identity Query is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The Identity & Access tab resolves user_id through users.employee_id into employee names. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not hardcode dropdown values when source tables can provide distinct values.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

29. Transitions UNION

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Job request, job card, and schedule histories are merged by UNION ALL with sub_source labels. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Real table/query	API meaning
audit_log	audit_log SELECT	All Actions
identity	user_role_history SELECT	Identity & Access
transitions	UNION ALL histories	Status Transitions

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Transitions UNION is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Job request, job card, and schedule histories are merged by UNION ALL with sub_source labels. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

CODE / CONFIG EXCERPT

```
SELECT ... FROM job_request_status_history
UNION ALL
SELECT ... FROM job_card_status_history
UNION ALL
SELECT ... FROM schedule_status_history
```

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

30. Detail Endpoint

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Detail fetch returns one row and safely handles JSON or plain notes. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Real table/query	API meaning
audit_log	audit_log SELECT	All Actions
identity	user_role_history SELECT	Identity & Access
transitions	UNION ALL histories	Status Transitions

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Detail Endpoint is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Detail fetch returns one row and safely handles JSON or plain notes. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

31. Filter Lookups

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Distinct action and entity-type lookups power dropdowns without hardcoding values. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Source	Real table/query	API meaning
audit_log	audit_log SELECT	All Actions
identity	user_role_history SELECT	Identity & Access
transitions	UNION ALL histories	Status Transitions

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Filter Lookups is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Distinct action and entity-type lookups power dropdowns without hardcoding values. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not treat notes as guaranteed JSON; fallback text is required.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

32. Service Transformation

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Service maps raw database rows into API-ready objects with summary, notes, and deep links. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Service Transformation is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Service maps raw database rows into API-ready objects with summary, notes, and deep links. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

33. Notes Parser

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. notes_json is parsed only when valid JSON; notes_text preserves legacy strings. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Notes Parser is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. notes_json is parsed only when valid JSON; notes_text preserves legacy strings. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

CODE / CONFIG EXCERPT

```
try { notes_json = JSON.parse(notes); }
catch { notes_text = notes; }
// Never throw only because legacy notes are plain text.
```

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

34. Summary Builder

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The list view summary explains what happened without forcing drawer expansion. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Summary Builder is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The list view summary explains what happened without forcing drawer expansion. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

35. Deep Links

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Known entity types receive frontend navigation paths while unknown types remain plain text. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Deep Links is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Known entity types receive frontend navigation paths while unknown types remain plain text. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

36. CSV Export

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Export reuses list filters and returns a compact audit-log CSV. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Concern	Rule	Locked behavior
Export	GET only	no write side effect
Cap	5000 rows	memory safe
Headers	X-Export-Rows/Capped	transparent client warning

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: CSV Export is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Export reuses list filters and returns a compact audit-log CSV. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

CODE / CONFIG EXCERPT

```
Content-Type: text/csv; charset=utf-8
Content-Disposition: attachment; filename="audit-log.csv"
X-Export-Rows: <rowCount>
X-Export-Capped: true|false
```

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

37. Export Cap

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Server-side cap prevents accidental large unbounded exports. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Concern	Rule	Locked behavior
Export	GET only	no write side effect
Cap	5000 rows	memory safe
Headers	X-Export-Rows/Capped	transparent client warning

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Export Cap is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Server-side cap prevents accidental large unbounded exports. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not use backslash for LIKE escape on this MariaDB setup; use vertical bar.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

38. Controller Layer

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Controller stays thin: call service, set response headers, handle next(error). The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Concern	Rule	Locked behavior
Export	GET only	no write side effect
Cap	5000 rows	memory safe
Headers	X-Export-Rows/Capped	transparent client warning

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Controller Layer is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Controller stays thin: call service, set response headers, handle next(error). The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

39. Express Route Order

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Static routes filters/export are registered before :id to avoid accidental param capture. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Concern	Rule	Locked behavior
Export	GET only	no write side effect
Cap	5000 rows	memory safe
Headers	X-Export-Rows/Capped	transparent client warning

FLOW / DIAGRAM

Route -> Validator -> Controller -> Service -> Repository -> MySQL SELECT

DETAILED DESCRIPTION

Detailed explanation: Express Route Order is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Static routes filters/export are registered before :id to avoid accidental param capture. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Route	/api/v1/audit	HTTP entry
Validator	Zod strict schemas	input safety
Service	canonical API shape	frontend stability
Repo	SELECT-only SQL	data correctness

TECHNICAL DEPTH

Technical depth: the backend uses a layered Express structure so each responsibility remains isolated. Validators reject bad query shapes, controllers keep HTTP concerns thin, services normalize data and build summaries, while repositories own all SQL. This keeps legacy table complexity away from frontend code and avoids accidental writes.

CODE / CONFIG EXCERPT

```
router.get('/filters', ...);
router.get('/export', ...);
router.get('/:id', ...);
// Static routes before :id.
```

IMPLEMENTATION CHECKLIST

- Keep all audit endpoints as GET.
- Keep static routes before /:id.
- Use prepared statements for SQL filters.
- Keep LIKE wildcard escaping before applying q search.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

40. Frontend Architecture

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Frontend contains API wrappers, React Query hooks, AuditViewer, and AuditDetailDrawer. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Layer	File	Responsibility
API	api/audit.js	HTTP calls
Hooks	useAuditLog.js	query lifecycle
Page	AuditViewer.jsx	main table
Drawer	AuditDetailDrawer.jsx	row details

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION
Detailed explanation: Frontend Architecture is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Frontend contains API wrappers, React Query hooks, AuditViewer, and AuditDetailDrawer. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH
Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY
Locked note: Do not create new audit tables for this phase. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST
• Keep filter state shareable through URL search params. • Keep export hidden without audit:export. • Keep drawer read-only with no mutation controls. • Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

41. API Wrapper

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. `api/audit.js` cleans params, calls GET endpoints, and downloads CSV as Blob. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Layer	File	Responsibility
API	<code>api/audit.js</code>	HTTP calls
Hooks	<code>useAuditLog.js</code>	query lifecycle
Page	<code>AuditViewer.jsx</code>	main table
Drawer	<code>AuditDetailDrawer.jsx</code>	row details

FLOW / DIAGRAM

API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: API Wrapper is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. `api/audit.js` cleans params, calls GET endpoints, and downloads CSV as Blob. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without `audit:export`.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

42. React Query Hooks

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. useAuditList, useAuditDetail, and useAuditFilters separate data orchestration from UI. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Layer	File	Responsibility
API	api/audit.js	HTTP calls
Hooks	useAuditLog.js	query lifecycle
Page	AuditViewer.jsx	main table
Drawer	AuditDetailDrawer.jsx	row details

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: React Query Hooks is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMC MIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. useAuditList, useAuditDetail, and useAuditFilters separate data orchestration from UI. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

43. AuditViewer Page Layout

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The main page combines title, tabs, filters, table, pagination, refresh, and export. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Layer	File	Responsibility
API	api/audit.js	HTTP calls
Hooks	useAuditLog.js	query lifecycle
Page	AuditViewer.jsx	main table
Drawer	AuditDetailDrawer.jsx	row details

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION
Detailed explanation: AuditViewer Page Layout is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The main page combines title, tabs, filters, table, pagination, refresh, and export. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH
Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY
Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST
• Keep filter state shareable through URL search params. • Keep export hidden without audit:export. • Keep drawer read-only with no mutation controls. • Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

44. URL-Synced Filters

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Filter state is reflected in search params so Super Admin can bookmark/share the view. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Layer	File	Responsibility
API	api/audit.js	HTTP calls
Hooks	useAuditLog.js	query lifecycle
Page	AuditViewer.jsx	main table
Drawer	AuditDetailDrawer.jsx	row details

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION
Detailed explanation: URL-Synced Filters is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Filter state is reflected in search params so Super Admin can bookmark/share the view. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH
Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY
Locked note: Do not hardcode dropdown values when source tables can provide distinct values. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST
• Keep filter state shareable through URL search params. • Keep export hidden without audit:export. • Keep drawer read-only with no mutation controls. • Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

45. Source Tabs UX

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. All Actions, Identity & Access, and Status Transitions create clean mental models. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Layer	File	Responsibility
API	api/audit.js	HTTP calls
Hooks	useAuditLog.js	query lifecycle
Page	AuditViewer.jsx	main table
Drawer	AuditDetailDrawer.jsx	row details

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION
Detailed explanation: Source Tabs UX is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMC MIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. All Actions, Identity & Access, and Status Transitions create clean mental models. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH
Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY
Locked note: Do not use backslash for LIKE escape on this MariaDB setup; use vertical bar. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST
• Keep filter state shareable through URL search params. • Keep export hidden without audit:export. • Keep drawer read-only with no mutation controls. • Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

46. Filter Strip UX

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Date, actor, action, entity type, entity ID, and search enable fast trace discovery. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
UI Part	Input/Output	Engineer value
Tabs	source	mental model
Filters	date/actor/action/entity/q	fast narrowing
Table	summary rows	scan quickly
Drawer	full detail	verify evidence

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: Filter Strip UX is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMC MIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Date, actor, action, entity type, entity ID, and search enable fast trace discovery. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

47. Data Table UX

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Rows show timestamp, actor, action, entity, summary, IP, and View action. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
UI Part	Input/Output	Engineer value
Tabs	source	mental model
Filters	date/actor/action/entity/q	fast narrowing
Table	summary rows	scan quickly
Drawer	full detail	verify evidence

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: Data Table UX is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Rows show timestamp, actor, action, entity, summary, IP, and View action. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not treat notes as guaranteed JSON; fallback text is required.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

48. Pagination UX

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Pagination keeps audit browsing controlled and avoids loading the full table at once. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
UI Part	Input/Output	Engineer value
Tabs	source	mental model
Filters	date/actor/action/entity/q	fast narrowing
Table	summary rows	scan quickly
Drawer	full detail	verify evidence

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: Pagination UX is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Pagination keeps audit browsing controlled and avoids loading the full table at once. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

49. Action Badges

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Semantic badges group JR, JC, schedule, user, employee, and equipment actions visually. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
UI Part	Input/Output	Engineer value
Tabs	source	mental model
Filters	date/actor/action/entity/q	fast narrowing
Table	summary rows	scan quickly
Drawer	full detail	verify evidence

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: Action Badges is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Semantic badges group JR, JC, schedule, user, employee, and equipment actions visually. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

50. Export UX

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Export CSV appears only when audit:export is granted. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
UI Part	Input/Output	Engineer value
Tabs	source	mental model
Filters	date/actor/action/entity/q	fast narrowing
Table	summary rows	scan quickly
Drawer	full detail	verify evidence

FLOW / DIAGRAM

API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: Export UX is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Export CSV appears only when audit:export is granted. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

51. AuditDetailDrawer

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The drawer gives read-only detail without navigating away from the table. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
UI Part	Input/Output	Engineer value
Tabs	source	mental model
Filters	date/actor/action/entity/q	fast narrowing
Table	summary rows	scan quickly
Drawer	full detail	verify evidence

FLOW / DIAGRAM

API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: AuditDetailDrawer is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The drawer gives read-only detail without navigating away from the table. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

52. Who / What / Where

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Drawer sections separate actor, entity, time, IP, user agent, and request id. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
UI Part	Input/Output	Engineer value
Tabs	source	mental model
Filters	date/actor/action/entity/q	fast narrowing
Table	summary rows	scan quickly
Drawer	full detail	verify evidence

FLOW / DIAGRAM

API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: Who / What / Where is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Drawer sections separate actor, entity, time, IP, user agent, and request id. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not hardcode dropdown values when source tables can provide distinct values. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

53. Transition Pills

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Status or role changes render as from-to pills for instant lifecycle understanding. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
UI Part	Input/Output	Engineer value
Tabs	source	mental model
Filters	date/actor/action/entity/q	fast narrowing
Table	summary rows	scan quickly
Drawer	full detail	verify evidence

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: Transition Pills is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMC MIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Status or role changes render as from-to pills for instant lifecycle understanding. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not use backslash for LIKE escape on this MariaDB setup; use vertical bar. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

54. JSON Detail View

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Pretty-printing notes_json makes compact audit payloads readable for Super Admin. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
UI Part	Input/Output	Engineer value
Tabs	source	mental model
Filters	date/actor/action/entity/q	fast narrowing
Table	summary rows	scan quickly
Drawer	full detail	verify evidence

FLOW / DIAGRAM
API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: JSON Detail View is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMC MIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Pretty-printing notes_json makes compact audit payloads readable for Super Admin. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

55. Empty and Error States

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Errors and no-match states are deliberately soft and non-mutating. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: Empty and Error States is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Errors and no-match states are deliberately soft and non-mutating. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not treat notes as guaranteed JSON; fallback text is required.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

56. Loading Behavior

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The drawer optimistically displays list data while detail data is fetched. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

API wrapper -> React Query hook -> AuditViewer -> Drawer -> Export CSV

DETAILED DESCRIPTION

Detailed explanation: Loading Behavior is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The drawer optimistically displays list data while detail data is fetched. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Page	AuditViewer	operator surface
Hook	useAuditLog	data lifecycle
Drawer	AuditDetailDrawer	row evidence
Export	downloadAuditCsv	offline review

TECHNICAL DEPTH

Technical depth: the frontend keeps the page fast and understandable by separating API wrappers, React Query hooks, page state, and drawer rendering. Filters are synchronized to the URL, export is permission-aware, and row detail is fetched lazily so the main table stays responsive.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Keep filter state shareable through URL search params.
- Keep export hidden without audit:export.
- Keep drawer read-only with no mutation controls.
- Render unknown entities as text instead of broken links.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

57. Accessibility Pass

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Labels, button roles, readable contrast, and structured sections support technical users. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Accessibility Pass is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Labels, button roles, readable contrast, and structured sections support technical users. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

58. Security Model

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Security comes from auth middleware, permission gates, strict validators, and absence of writes. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Security Model is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Security comes from auth middleware, permission gates, strict validators, and absence of writes. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

59. Zero-Write Proof

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The smoke test captures baseline and after counts for every source table. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Zero-Write Proof is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMC MIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The smoke test captures baseline and after counts for every source table. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

60. Filter Cache

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Source filter dropdowns use short in-process caching to avoid repeated grouping queries. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Filter Cache is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Source filter dropdowns use short in-process caching to avoid repeated grouping queries. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not hardcode dropdown values when source tables can provide distinct values.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

61. MariaDB LIKE Escape Rule

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

During implementation, a subtle MariaDB issue appeared: backslash as a SQL LIKE escape produced a parser failure. The locked fix is to use the vertical bar character as the escape marker and transform user input before LIKE matching.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: MariaDB LIKE Escape Rule is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Use | as SQL LIKE escape char on MariaDB; avoid backslash escape failure. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

CODE / CONFIG EXCERPT

```
WHERE action LIKE ? ESCAPE '|'
// q='JR_' becomes pattern '%JR|_%'
// Prevents '_' from acting as SQL wildcard.
```

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

62. Smoke Matrix Overview

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The verification file checks API shape, filters, RBAC, export, pagination, and data immutability. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
A1	List newest-first	PASS
A2	Actor filter	PASS
A3	Action filter	PASS
A4	Entity type filter	PASS

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Smoke Matrix Overview is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The verification file checks API shape, filters, RBAC, export, pagination, and data immutability. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

63. Smoke A1-A4

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. A1 to A4 validate newest-first list, actor filter, action filter, and entity type filter. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

Smoke ID	Assertion	Result
A3	Action filter	PASS
A4	Entity type filter	PASS
A5	Entity ID filter	PASS
A6	Free-text q	PASS

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Smoke A1-A4 is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. A1 to A4 validate newest-first list, actor filter, action filter, and entity type filter. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not treat notes as guaranteed JSON; fallback text is required.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

64. Smoke A5-A8

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. A5 to A8 validate entity ID, free text, identity source, and transitions union. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

Smoke ID	Assertion	Result
A5	Entity ID filter	PASS
A6	Free-text q	PASS
A7	Identity source	PASS
A8	Transitions UNION	PASS

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Smoke A5-A8 is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. A5 to A8 validate entity ID, free text, identity source, and transitions union. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

65. Smoke A9-A12

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. A9 to A12 validate detail, dropdowns, CSV export, and RBAC boundaries. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

Smoke ID	Assertion	Result
A7	Identity source	PASS
A8	Transitions UNION	PASS
A9	Detail and notes	PASS
A10	Filter dropdowns	PASS

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Smoke A9-A12 is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. A9 to A12 validate detail, dropdowns, CSV export, and RBAC boundaries. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

66. Smoke A13-A16

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. A13 to A16 validate pagination, page uniqueness, row-count proof, and direct SQL cross-check. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

Smoke ID	Assertion	Result
A9	Detail and notes	PASS
A10	Filter dropdowns	PASS
A11	CSV export	PASS
A12	RBAC	PASS

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Smoke A13-A16 is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. A13 to A16 validate pagination, page uniqueness, row-count proof, and direct SQL cross-check. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

67. Final Smoke Output

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

The final smoke matrix completed with all checks green: 20 passed and 0 failed. This is the operational proof that the page, API, export, RBAC, pagination, filters, and source-table immutability all work together.

Smoke ID	Assertion	Result
A11	CSV export	PASS
A12	RBAC	PASS
A13	Pagination math	PASS
A14	Page uniqueness	PASS

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Final Smoke Output is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. 20/20 smoke checks passed with zero failures. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

68. READ-ONLY Count Proof

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Counts remained unchanged across audit_log plus all history tables after smoke execution. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

Smoke ID	Assertion	Result
A13	Pagination math	PASS
A14	Page uniqueness	PASS
A15	Read-only counts	PASS
A16	Direct SELECT cross-check	PASS

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: READ-ONLY Count Proof is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Counts remained unchanged across audit_log plus all history tables after smoke execution. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not hardcode dropdown values when source tables can provide distinct values.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

69. Direct SELECT Cross-Check

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. API row data was verified against direct SELECT output for correctness. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

Smoke ID	Assertion	Result
A15	Read-only counts	PASS
A16	Direct SELECT cross-check	PASS

FLOW / DIAGRAM

Run smoke -> Assert RBAC -> Assert filters -> Assert row counts -> Seal

DETAILED DESCRIPTION

Detailed explanation: Direct SELECT Cross-Check is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. API row data was verified against direct SELECT output for correctness. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Smoke	A1-A16	behavior proof
RBAC	401/403/200	access proof
Counts	before/after	read-only proof
SQL cross-check	direct SELECT	data proof

TECHNICAL DEPTH

Technical depth: verification is not a casual checklist. The smoke script exercises live authenticated routes, checks narrowing filters, validates source-tab behavior, proves RBAC, confirms CSV generation, and compares counts before and after the run to prove that audit evidence was not modified.

LOCKED NOTE / MEMORY

Locked note: Do not use backslash for LIKE escape on this MariaDB setup; use vertical bar.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Run smoke_phase14.js after backend changes.
- Verify row counts remain unchanged.
- Check RBAC with anonymous, View-Only, and Super Admin users.
- Keep cross-select verification in the smoke matrix.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

70. Deployment Flow

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Apply migration, run backend, open Super Admin, verify /audit, then run smoke. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Deployment Flow is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Apply migration, run backend, open Super Admin, verify /audit, then run smoke. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

CODE / CONFIG EXCERPT

```
cd DATABASE/phase3 && node runner/run-migrations.js
cd BE && npm run dev
cd BE && node db/discovery/smoke_phase14.js
```

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

71. Migration Re-run Safety

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. INSERT IGNORE makes migration 600 idempotent and safe to re-run. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Migration Re-run Safety is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. INSERT IGNORE makes migration 600 idempotent and safe to re-run. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not treat notes as guaranteed JSON; fallback text is required.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

72. Server Mount

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. server.js mounts audit routes under the API base path after procurement. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Server Mount is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. server.js mounts audit routes under the API base path after procurement. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

73. Frontend Route

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. App.jsx replaces placeholder /audit route with the real AuditViewer page. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Frontend Route is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. App.jsx replaces placeholder /audit route with the real AuditViewer page. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

74. Backend Files Matrix

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The backend module is contained in validators, repo, service, controller, and routes. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Migration	600_phase14_audit_permissions.sql	permission seed
Validators	audit.validators.js	strict query/path schemas
Repository	audit.repo.js	SELECT-only DAL
Service	audit.service.js	canonical response shaping
Controller/Routes	audit.controller.js / audit.routes.js	HTTP layer

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Backend Files Matrix is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The backend module is contained in validators, repo, service, controller, and routes. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

75. Frontend Files Matrix

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Frontend changes are split across api, hooks, page, drawer, permissions, and app route. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
API	lib/api/audit.js	GET wrappers + CSV Blob
Hooks	lib/hooks/useAuditLog.js	React Query hooks
Page	pages/audit/AuditViewer.jsx	tabs/filter/table UI
Drawer	pages/audit/AuditDetailDrawer.jsx	read-only detail panel
Navigation	permissions.js + App.jsx	sidebar + route

FLOW / DIAGRAM
Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Frontend Files Matrix is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Frontend changes are split across api, hooks, page, drawer, permissions, and app route. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

76. Code Appendix: Validators

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Validators define source enum, list query, detail params, filters, and export query. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Code Appendix: Validators is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Validators define source enum, list query, detail params, filters, and export query. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

CODE / CONFIG EXCERPT

```
const SOURCES = ['audit_log','identity','transitions'];
const listQuerySchema = z.object({...}).strict();
```

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

77. Code Appendix: Repo Helpers

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Repo helpers implement time-range boundaries and LIKE wildcard escaping. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Code Appendix: Repo Helpers is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Repo helpers implement time-range boundaries and LIKE wildcard escaping. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

CODE / CONFIG EXCERPT

```
function likeContains(q) {
  return '%' + q.replace(/[%_]/g, m => '|' + m) + '%';
}
```

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

78. Code Appendix: Transitions UNION

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The union standardizes id, occurred_at, action, entity, actor, and status fields. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Code Appendix: Transitions UNION is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The union standardizes id, occurred_at, action, entity, actor, and status fields. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

CODE / CONFIG EXCERPT

```
source_table = 'job_request' | 'job_card' | 'schedule'
sub_source travels with each transition row.
```

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

79. Code Appendix: Service API Shape

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. toApi produces the canonical response consumed by the frontend. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Code Appendix: Service API Shape is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. toApi produces the canonical response consumed by the frontend. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

CODE / CONFIG EXCERPT

```
return { id, source, sub_source, occurred_at, action,
  entity_type, entity_id, actor_employee_id, notes_json, notes_text };
```

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

80. Code Appendix: CSV Export

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. CSV export escapes values and includes audit-oriented header columns. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Code Appendix: CSV Export is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. CSV export escapes values and includes audit-oriented header columns. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

CODE / CONFIG EXCERPT

```
for (const row of items) {
  lines.push(cols.map(csvEscape).join(','));
}
```

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

81. Code Appendix: AuditViewer

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. AuditViewer manages tabs, filters, URL sync, table, pagination, refresh, and export. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Code Appendix: AuditViewer is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. AuditViewer manages tabs, filters, URL sync, table, pagination, refresh, and export. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

CODE / CONFIG EXCERPT

```
<AuditViewer />
// title + tabs + filters + DataTable + Pagination + Drawer
```

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

82. Code Appendix: Drawer

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. AuditDetailDrawer renders read-only row detail and transition visualizations. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Code Appendix: Drawer is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. AuditDetailDrawer renders read-only row detail and transition visualizations. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

CODE / CONFIG EXCERPT

```
<AuditDetailDrawer row={openRow} onClose={...} />
// Who / What / Where / Transition / Details
```

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

83. Testing Guide

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Run the smoke script from BE and expect 20 passed, 0 failed. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Apply migration -> Restart backend -> Open /audit -> Run smoke -> Document

DETAILED DESCRIPTION

Detailed explanation: Testing Guide is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Run the smoke script from BE and expect 20 passed, 0 failed. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

84. Troubleshooting 401/403

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. 401 means unauthenticated; 403 means authenticated but missing audit permissions. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Detect issue -> Map cause -> Apply locked rule -> Verify again

DETAILED DESCRIPTION

Detailed explanation: Troubleshooting 401/403 is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. 401 means unauthenticated; 403 means authenticated but missing audit permissions. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not hardcode dropdown values when source tables can provide distinct values.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

85. Troubleshooting LIKE Escape

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Use | in LIKE escape logic because backslash failed on MariaDB during implementation. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Detect issue -> Map cause -> Apply locked rule -> Verify again

DETAILED DESCRIPTION

Detailed explanation: Troubleshooting LIKE Escape is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Use | in LIKE escape logic because backslash failed on MariaDB during implementation. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not use backslash for LIKE escape on this MariaDB setup; use vertical bar.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

86. Edge Cases

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Null notes, plain text notes, missing employee names, unknown entity types, and empty result sets are handled. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Detect issue -> Map cause -> Apply locked rule -> Verify again

DETAILED DESCRIPTION

Detailed explanation: Edge Cases is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Null notes, plain text notes, missing employee names, unknown entity types, and empty result sets are handled. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

87. Performance Review

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Pagination, indexes, cache, and export caps keep the viewer efficient. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Detect issue -> Map cause -> Apply locked rule -> Verify again

DETAILED DESCRIPTION

Detailed explanation: Performance Review is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMC MIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Pagination, indexes, cache, and export caps keep the viewer efficient. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not treat notes as guaranteed JSON; fallback text is required.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

88. Governance Review

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Super Admin-only audit access prevents unnecessary exposure of sensitive trace data. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Detect issue -> Map cause -> Apply locked rule -> Verify again

DETAILED DESCRIPTION

Detailed explanation: Governance Review is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Super Admin-only audit access prevents unnecessary exposure of sensitive trace data. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

89. Audit Integrity

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The viewer observes audit trails but does not mutate them or create recursive reads. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Detect issue -> Map cause -> Apply locked rule -> Verify again

DETAILED DESCRIPTION

Detailed explanation: Audit Integrity is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The viewer observes audit trails but does not mutate them or create recursive reads. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

90. Future Extension

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Future roles can receive audit permissions through a new migration if governance policy changes. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Detect issue -> Map cause -> Apply locked rule -> Verify again

DETAILED DESCRIPTION

Detailed explanation: Future Extension is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMIS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Future roles can receive audit permissions through a new migration if governance policy changes. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

91. Phase Relationship

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Phase 14 consumes audit traces created by prior sealed phases and does not reopen them. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Detect issue -> Map cause -> Apply locked rule -> Verify again

DETAILED DESCRIPTION

Detailed explanation: Phase Relationship is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Phase 14 consumes audit traces created by prior sealed phases and does not reopen them. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

92. Locked Decisions D1-D5

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. D1-D5 lock notes strategy, source aliasing, transitions union, actor resolution, and LIKE escape. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Decision	Locked Rule	Reason
D1/D6	notes_json or notes_text	legacy safe
D3/D8	sub_source + export cap	safe lookup/download
D5/D10	LIKE escape + new perms	MariaDB and RBAC correctness

FLOW / DIAGRAM
Review decisions -> Lock behavior -> Preserve sealed state -> Future additive change only

DETAILED DESCRIPTION
Detailed explanation: Locked Decisions D1-D5 is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. D1-D5 lock notes strategy, source aliasing, transitions union, actor resolution, and LIKE escape. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH
Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY
Locked note: Do not hardcode dropdown values when source tables can provide distinct values. Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST
• Treat Phase 14 as sealed. • Use a new migration for permission expansion. • Document every future change with smoke proof. • Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

93. Locked Decisions D6-D10

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. D6-D10 lock case-insensitive search, filters cache, export cap, no read-audit recursion, and new permissions. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Decision	Locked Rule	Reason
D1/D6	notes_json or notes_text	legacy safe
D3/D8	sub_source + export cap	safe lookup/download
D5/D10	LIKE escape + new perms	MariaDB and RBAC correctness

FLOW / DIAGRAM

Review decisions -> Lock behavior -> Preserve sealed state -> Future additive change only

DETAILED DESCRIPTION

Detailed explanation: Locked Decisions D6-D10 is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. D6-D10 lock case-insensitive search, filters cache, export cap, no read-audit recursion, and new permissions. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not use backslash for LIKE escape on this MariaDB setup; use vertical bar.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

94. Acceptance Criteria

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The page is visible, secure, useful, fast, exportable, and read-only. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Review decisions -> Lock behavior -> Preserve sealed state -> Future additive change only

DETAILED DESCRIPTION

Detailed explanation: Acceptance Criteria is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The page is visible, secure, useful, fast, exportable, and read-only. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not bypass validators; strict query validation protects the module surface.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

95. Executive Explanation

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. For reviewers, Phase 14 turns raw technical audit trails into an operational governance console. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Review decisions -> Lock behavior -> Preserve sealed state -> Future additive change only

DETAILED DESCRIPTION

Detailed explanation: Executive Explanation is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. For reviewers, Phase 14 turns raw technical audit trails into an operational governance console. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not treat notes as guaranteed JSON; fallback text is required.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

96. Mentor / Judge Explanation

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. The value is traceability, accountability, and confidence in a production-grade CMCMS environment. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Review decisions -> Lock behavior -> Preserve sealed state -> Future additive change only

DETAILED DESCRIPTION

Detailed explanation: Mentor / Judge Explanation is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. The value is traceability, accountability, and confidence in a production-grade CMCMS environment. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not create new audit tables for this phase.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

97. Developer Handover

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Developers should change only via additive migrations and preserve the read-only doctrine. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Review decisions -> Lock behavior -> Preserve sealed state -> Future additive change only

DETAILED DESCRIPTION

Detailed explanation: Developer Handover is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Developers should change only via additive migrations and preserve the read-only doctrine. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not widen access beyond SUPER_ADMIN without a new governance migration.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

98. Revision Notes

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. This document seals the design state and records the implementation decisions for future work. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Review decisions -> Lock behavior -> Preserve sealed state -> Future additive change only

DETAILED DESCRIPTION

Detailed explanation: Revision Notes is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. This document seals the design state and records the implementation decisions for future work. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not add write-side effects to read/export actions.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

99. Final Lock

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This page records the Phase 14 Audit Log Viewer design in the DS-style documentation format. Phase 14 is considered delivered and should not be reopened without explicit instruction. The goal is not only to describe the feature, but to preserve the engineering logic so a future developer can continue without breaking the sealed state.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Review decisions -> Lock behavior -> Preserve sealed state -> Future additive change only

DETAILED DESCRIPTION

Detailed explanation: Final Lock is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Phase 14 is considered delivered and should not be reopened without explicit instruction. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not remove legacy audit_log:read abruptly; it remains historically harmless.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.

100. Seal Page

STATUS: SEALED	SMOKE: 20/20 GREEN	ACCESS: SA-only	MODE: READ-ONLY
----------------	--------------------	-----------------	-----------------

This is the final seal page. Phase 14 is delivered as a Super Admin-only, read-only, tested, documented, and locked Audit Log Viewer. Future work must treat this state as sealed unless the project owner explicitly reopens it.

DS Block	Description	Phase-14 Meaning
Block	Meaning	Locked Output
Design	What we built	Audit visibility
Security	Who can use it	SUPER_ADMIN only by default
Verification	How we proved it	20/20 GREEN

FLOW / DIAGRAM

Review decisions -> Lock behavior -> Preserve sealed state -> Future additive change only

DETAILED DESCRIPTION

Detailed explanation: Seal Page is treated as a production documentation unit, not a loose note. The Phase 14 record must explain the requirement, the table or code surface involved, the permission boundary, the runtime behavior, and the verification proof. In CMCMS terms, this page helps a future developer answer four questions quickly: what does this part do, which legacy data does it touch, what must never change, and how was it proven safe. Final DS-style seal: Phase 14 Audit Log Viewer is locked, documented, and verified. The writing style intentionally combines simple explanation with deeper implementation detail, because the module may be reviewed by developers, admins, mentors, and government-grade technical users.

Concept	Connected Element	Why It Matters
Memory	locked decisions	future safety
Change path	additive migration	governance
Risk	scope creep	avoid mutation
Seal	Phase 14 locked	handover

TECHNICAL DEPTH

Technical depth: future changes must be additive and governance-driven. New roles should receive audit access only through a new migration. New source tables should be attached through a source-tab model or a controlled union. Existing sealed behavior should not be silently rewritten.

LOCKED NOTE / MEMORY

Locked note: Do not hardcode dropdown values when source tables can provide distinct values.
Implementation memory: Audit Log Viewer remains a read-only governance surface over existing operational evidence.

IMPLEMENTATION CHECKLIST

- Treat Phase 14 as sealed.
- Use a new migration for permission expansion.
- Document every future change with smoke proof.
- Never reopen prior phases silently.

Acceptance mark: This page is part of the locked Phase 14 documentation pack and preserves the DS-style trace from design to verification.